



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente
Ingeniería en Desarrollo de Software

Asignatura: **Administración de Bases de Datos**

Unidad #5

Fecha: Del 27 de octubre al 02 de noviembre 2025

Tema: Seguridad.

Introducción:

Este material tiene como propósito ofrecerte una visión clara y aplicada sobre tres aspectos esenciales en la administración de bases de datos enfocados en el ámbito de la seguridad como: métodos de respaldo, recuperación ante fallos y migración de datos, enfocados principalmente en PostgreSQL. Primero, se describen distintas estrategias de respaldo, desde las más simples hasta esquemas avanzados como el archivado continuo y la recuperación en un momento dado (PITR). Luego, se abordan los comandos clave para restaurar la base de datos, ya sea a partir de volcados lógicos o copias físicas, asegurando la continuidad operativa ante errores o pérdidas. Finalmente, se explica cómo realizar procesos de migración, incluyendo la exportación e importación de datos entre servidores, un procedimiento común en actualizaciones, traslados o réplicas de entornos.

Sección 1: Métodos de respaldo de un DBMS.

Como en todo sistema que contiene información valiosa, es fundamental realizar copias de seguridad periódicas de las bases de datos por ejemplo en PostgreSQL. Si bien el procedimiento es relativamente sencillo, resulta crucial comprender con claridad las técnicas involucradas y los supuestos sobre los que se sustentan. En PostgreSQL, existen tres enfoques principales para la realización de respaldos:

1. El volcado en formato SQL (SQL Dump).
2. La copia a nivel del sistema de archivos.
3. El archivado continuo (WAL Archiving).

Cada uno de estos métodos posee ventajas, limitaciones y casos de uso particulares. A continuación, se desarrollan en detalle:

1.Volcado de SQL (SQL Dump)

El método de volcado mediante SQL en PostgreSQL consiste en generar un archivo de texto que contiene instrucciones SQL capaces de reconstruir una base de datos exactamente como estaba en el momento en que se realizó el volcado. Es decir, este archivo puede contener sentencias CREATE TABLE, INSERT, ALTER, COMMENT, entre muchas otras, que permiten recrear la estructura y los datos de la base de datos de forma fiel. Esta técnica resulta particularmente útil cuando se desea migrar bases de datos, compartir datos entre ambientes (desarrollo, pruebas, producción) o simplemente realizar copias de seguridad con fines de recuperación ante fallos.



Para lograr esto, PostgreSQL proporciona una utilidad llamada `pg_dump`, una herramienta de línea de comandos incluida en todas las instalaciones estándar del sistema gestor de bases de datos. Su función es generar el archivo de volcado en formato texto o en formatos especiales que permiten una gestión más avanzada de la restauración. El uso más básico de `pg_dump` se ve así:

`pg_dump nombre_base_de_datos > archivo_dump.sql`

Este comando ejecuta un volcado de la base de datos especificada y guarda el resultado en un archivo llamado `archivo_dump.sql`. Como puede observarse, `pg_dump` genera su salida a través de la salida estándar del sistema (`stdout`), lo cual permite canalizar (pipear) su resultado hacia otros programas, como herramientas de compresión o scripts personalizados. Esta característica es especialmente útil cuando se trabaja en entornos Unix o Linux, donde el uso de tuberías es una práctica común y poderosa.

Aunque este comando básico produce un archivo de texto plano, `pg_dump` también puede generar volcados en otros formatos, como el formato personalizado (`custom`), el formato de directorio (`directory`) y el formato `tar`. Estos formatos no solo permiten compresión interna y división en múltiples archivos, sino también la posibilidad de restaurar los objetos de la base de datos de forma selectiva o en paralelo, lo cual es de gran utilidad en bases de datos de gran tamaño o cuando se requiere una restauración controlada y eficiente.

Características de `pg_dump` como herramienta cliente

Es importante destacar que `pg_dump` no es una herramienta que funcione directamente dentro del servidor de base de datos. Al contrario, es una aplicación cliente que se conecta al servidor usando los mismos mecanismos que cualquier otra herramienta como `psql` o `pgAdmin`. En otras palabras, `pg_dump` puede ejecutarse desde cualquier equipo que tenga conectividad con el servidor PostgreSQL y los permisos necesarios para acceder a los datos. Esto lo hace sumamente versátil, ya que permite realizar respaldos incluso desde máquinas remotas sin necesidad de tener acceso físico al servidor.

Sin embargo, esta característica también impone ciertas limitaciones. `pg_dump` no posee privilegios especiales por defecto. Es decir, el usuario que ejecuta el volcado debe tener privilegios suficientes para acceder a los objetos que desea respaldar. Si el objetivo es hacer una copia de seguridad completa de una base de datos, lo más común es ejecutar el comando con un usuario con rol de superusuario o con privilegios de lectura sobre todas las tablas, funciones, secuencias, vistas y demás objetos que componen la base de datos.

Aun así, si solo se desea respaldar partes específicas de la base de datos (como un esquema o una tabla concreta), `pg_dump` ofrece opciones como `-n` para especificar el esquema y `-t` para indicar una tabla. Estas opciones permiten realizar respaldos más granulares, lo cual es útil para ambientes de desarrollo o para aplicaciones que trabajan con estructuras desacopladas.

Ejemplo:

Supongamos que tienes una base de datos llamada `empresa_db` y dentro de ella existe un esquema llamado `recursos_humanos` que contiene una tabla llamada `empleados`. Si deseas realizar un respaldo solo de esa tabla específica, puedes usar el siguiente comando:

`pg_dump -U usuario -d empresa_db -n recursos_humanos -t recursos_humanos.empleados -f empleados_backup.sql`



Donde:

- **-U usuario:** Es el nombre del usuario con permisos suficientes para acceder a la base de datos.
- **-d empresa_db:** Es el nombre de la base de datos de la cual se realizará el respaldo.
- **-n recursos_humanos:** Especifica el esquema que contiene la tabla.
- **-t recursos_humanos.empleados:** Indica la tabla específica que se desea respaldar es buena práctica anteponer el esquema.
- **-f empleados_backup.sql:** Define el nombre del archivo donde se guardará el respaldo.

Este tipo de respaldo es útil si solo se está trabajando con una parte específica del sistema, como un módulo de recursos humanos, y no es necesario respaldar toda la base de datos.

Opciones de conexión

Para especificar el servidor al que debe conectarse `pg_dump`, se utilizan las opciones `-h` y `-p` para el host y el puerto, respectivamente. Si no se especifica ningún valor, el comando intentará conectarse a través del host local (por defecto) o utilizando los valores definidos en las variables de entorno `PGHOST` y `PGPORT`.

Del mismo modo, el nombre de usuario por defecto será el mismo que el del usuario del sistema operativo que ejecuta el comando. Si se desea utilizar otro nombre de usuario, se puede usar la opción `-U`, o bien establecer la variable de entorno `PGUSER`.

Cabe recordar que todas las conexiones realizadas por `pg_dump` están sujetas a los mecanismos normales de autenticación del servidor PostgreSQL, definidos en el archivo `pg_hba.conf`. Esto significa que el comando puede solicitar contraseña, autenticarse mediante certificados o incluso denegar el acceso, dependiendo de la configuración de seguridad del servidor.

Ejemplo de conexión remota con `pg_dump`:

Supongamos que deseas realizar un respaldo de una base de datos llamada `ventas_db` que se encuentra en un servidor remoto con la IP `192.168.1.100`, escuchando en el puerto `5432`, y quieres conectarte como el usuario `admin`.

Puedes usar el siguiente comando:

```
pg_dump -h 192.168.1.100 -p 5432 -U admin -d ventas_db -f respaldo_ventas.sql
```

Donde:

- **-h 192.168.1.100:** especifica el host (IP o nombre del servidor) donde se encuentra la base de datos.
- **-p 5432:** indica el puerto en el que escucha el servidor PostgreSQL (5432 es el valor por defecto).
- **-U admin:** usa el usuario `admin` para la conexión.
- **-d ventas_db:** nombre de la base de datos que se va a respaldar.
- **-f respaldo_ventas.sql:** define el nombre del archivo donde se guardará el volcado.

Nota: Si la autenticación del servidor lo requiere, el sistema pedirá la contraseña del usuario `admin` al ejecutar el comando.



Ventajas clave de pg_dump

Una de las principales ventajas del uso de pg_dump frente a otros métodos de copia de seguridad más bajos (como los respaldos físicos o el archivado de WAL) es su compatibilidad entre versiones. Es decir, los archivos generados con pg_dump suelen poder ser restaurados en versiones futuras de PostgreSQL, incluso si hubo cambios en la representación interna de datos o en el motor de almacenamiento.

Este nivel de portabilidad convierte a pg_dump en una herramienta fundamental cuando se desea migrar una base de datos entre servidores, versiones o incluso arquitecturas de hardware distintas (por ejemplo, de un servidor de 32 bits a uno de 64 bits).

Además, los volcados creados por pg_dump son internamente consistentes. Esto significa que el archivo representa una instantánea precisa de la base de datos en el momento exacto en que comenzó la operación de volcado. Mientras se ejecuta, pg_dump no bloquea otras operaciones en la base de datos, salvo aquellas que requieran bloqueos exclusivos, como algunas formas de ALTER TABLE, esto permite realizar respaldos sin interrumpir el servicio.

Usando pg_dumpall para volcar un clúster completo

A diferencia de pg_dump, que se enfoca en una sola base de datos, PostgreSQL ofrece la utilidad pg_dumpall para realizar respaldos completos de un clúster de bases de datos. Este comando incluye no solo todas las bases de datos presentes en el servidor, sino también información global como:

- Roles (usuarios y grupos)
- Definiciones de privilegios
- Espacios de tablas (tablespaces)
- Configuraciones a nivel de clúster

El uso básico de este comando es:

```
pg_dumpall > archivo_volcado_completo.sql
```

Este archivo de volcado puede ser restaurado utilizando psql de la siguiente forma:

```
psql -f archivo_volcado_completo.sql postgres
```

En general, al restaurar un volcado completo con pg_dumpall, se recomienda usar una base de datos existente como postgres como punto de entrada, especialmente si se está restaurando en un clúster recién inicializado. Es indispensable tener privilegios de superusuario durante la restauración, ya que de lo contrario no se podrá recrear la información global como los roles o los espacios de tabla.

Es importante tener en cuenta que, aunque cada base de datos individual se vuelca de forma consistente, pg_dumpall no garantiza que todas las bases estén sincronizadas entre sí en términos de tiempo. Esto se debe a que realiza los volcados secuencialmente y no como una sola operación atómica.

Si solo se desea respaldar la información global del clúster (roles, privilegios, etc.), se puede utilizar la opción:

```
pg_dumpall --globals-only > globals.sql
```

Esto resulta útil cuando se combinan respaldos individuales por base de datos (usando pg_dump) con un respaldo de la configuración global del clúster.



Manejo de grandes volúmenes de datos

Cuando se trabaja con bases de datos grandes, es común enfrentar limitaciones impuestas por el sistema operativo, especialmente en lo referente al tamaño máximo de los archivos. Afortunadamente, existen estrategias bien conocidas que permiten superar estas barreras utilizando herramientas estándar de Unix/Linux, aprovechando que `pg_dump` puede emitir la salida por la salida estándar.

Una de las estrategias más comunes es comprimir el archivo de volcado al vuelo mientras se genera. Esto se puede lograr utilizando herramientas como `gzip`, por ejemplo:

```
pg_dump nombrebd | gzip > respaldo.sql.gz
```

Y para restaurar:

```
gunzip -c respaldo.sql.gz | psql nombrebd
```

O de forma alternativa:

```
zcat respaldo.sql.gz | psql nombrebd
```

División en múltiples archivos

Otra técnica útil es la división del archivo de salida en bloques más pequeños utilizando el comando `split`. Esto puede ser necesario cuando se trabaja en sistemas de archivos que imponen límites estrictos al tamaño de los archivos.

Por ejemplo, para dividir el volcado en archivos de 2 GB:

```
pg_dump nombrebd | split -b 2G - respaldo_parte_
```

Para restaurar:

```
cat respaldo_parte_* | psql nombrebd
```

Si se utiliza GNU `split`, es posible combinar división y compresión:

```
pg_dump nombrebd | split -b 2G --filter='gzip > $FILE.gz'
```

Luego, para restaurar las partes comprimidas:

```
zcat respaldo_parte_*.gz | psql nombrebd
```

Formato de volcado personalizado

Una opción más avanzada y recomendada para grandes bases de datos es utilizar el formato personalizado de `pg_dump`. Este formato no solo reduce el tamaño del archivo gracias a la compresión integrada, sino que también permite restauraciones selectivas y paralelas con la herramienta `pg_restore`:

- Comando para crear un volcado personalizado:
 - `pg_dump -Fc nombrebd > respaldo.dump`
- Y para restaurar con varios hilos (por ejemplo, 4):
 - `pg_restore -j 4 -d nombrebd respaldo.dump`

Este enfoque permite una restauración significativamente más rápida y controlada, especialmente útil en entornos donde el tiempo de inactividad debe minimizarse.



2.Copia de seguridad a nivel del sistema de archivos

Una estrategia alternativa para realizar copias de seguridad en PostgreSQL consiste en copiar directamente los archivos físicos que utiliza el sistema para almacenar la base de datos. Estos archivos se encuentran dentro del directorio de datos del clúster, cuya ubicación está definida al momento de la inicialización del clúster y descrita en la documentación oficial.

Para ejecutar esta copia de seguridad, se pueden utilizar herramientas estándar del sistema operativo, como tar, por ejemplo:

```
tar -cf backup.tar /usr/local/pgsql/data
```

Donde:

- **tar:** es una herramienta de archivado común en sistemas Unix/Linux.
- **-c:** crea un nuevo archivo tar.
- **-f backup.tar:** especifica el nombre del archivo de salida (backup.tar).
- **/usr/local/pgsql/data:** es el directorio que contiene todos los archivos físicos de la base de datos.

Este comando genera un archivo comprimido (o empaquetado) con todo el contenido del directorio de datos del clúster de PostgreSQL.

Sin embargo, este método presenta algunas limitaciones críticas que deben considerarse antes de adoptarlo como mecanismo principal de respaldo. La más importante es que el servidor de base de datos debe estar detenido para garantizar que los archivos copiados estén en un estado consistente. Intentar realizar una copia sin apagar el servidor puede resultar en archivos corruptos o inconsistentes, ya que herramientas como tar no garantizan una instantánea atómica del sistema de archivos. Además, PostgreSQL mantiene información interna en búferes de memoria que aún no se ha escrito a disco, lo cual puede dejar fuera cambios recientes si el servidor está en ejecución durante la copia.

Apagar el servidor también es un requisito en el proceso de restauración. Intentar copiar los archivos nuevamente al sistema mientras PostgreSQL está activo puede comprometer la integridad del clúster. Es decir, tanto para realizar como para aplicar este tipo de backup, se requiere detener completamente el servidor.

Otra consideración importante es que no es posible seleccionar archivos individuales del sistema de archivos para restaurar partes específicas de la base de datos, como una tabla o incluso una base de datos específica dentro del clúster. Esto se debe a que la estructura física de los datos en PostgreSQL depende de múltiples archivos relacionados, entre ellos los archivos WAL (Write-Ahead Logging), que contienen información sobre el estado de las transacciones. Por ejemplo, los archivos ubicados en pg_xact/ mantienen el estado de confirmación de todas las transacciones. Restaurar solo el archivo de una tabla sin los archivos WAL asociados puede dañar completamente el clúster, afectando incluso a las bases de datos no relacionadas con esa tabla.

Por esta razón, las copias de seguridad a nivel de sistema de archivos solo son viables para respaldar y restaurar todo el clúster de PostgreSQL, y no deben usarse para recuperar elementos individuales.



Instantáneas consistentes con soporte del sistema de archivos

Una alternativa más moderna y segura es aprovechar la funcionalidad de "instantáneas" o snapshots del sistema de archivos, si este lo permite. Sistemas como ZFS, LVM o Btrfs ofrecen soporte para crear instantáneas consistentes del volumen que contiene el directorio de datos. Este enfoque permite capturar el estado exacto del sistema en un momento determinado, incluso con el servidor en ejecución.

El procedimiento típico consiste en:

1. Forzar un CHECKPOINT para escribir a disco todas las transacciones pendientes.
2. Crear una instantánea del volumen donde reside el directorio de datos.
3. Copiar el contenido completo de dicha instantánea a un medio de respaldo.
4. Eliminar la instantánea una vez que la copia ha finalizado.

Aunque este método es compatible con sistemas en ejecución, la copia resultante reflejará un estado en el que el servidor no fue cerrado correctamente. Por lo tanto, al restaurar PostgreSQL asumirá que ocurrió una falla inesperada y ejecutará una recuperación automática utilizando los registros WAL. Este comportamiento es esperado y no debería representar un problema, siempre que se incluyan también los archivos WAL en la copia de seguridad.

Es fundamental copiar todo el directorio de datos, no solo partes de él. Esto incluye subdirectorios como `pg_wal`, `pg_xact`, `pg_multixact`, entre otros. También se debe tener en cuenta que, si los datos están distribuidos en múltiples volúmenes o sistemas de archivos, como cuando los WAL y los datos están en discos separados, entonces todas las instantáneas deben ser tomadas de forma simultánea. Si no es posible hacerlo, la copia puede quedar en un estado inconsistente.

Alternativa con rsync y tiempo de inactividad mínimo

Cuando no es posible usar snapshots, una alternativa válida es utilizar `rsync`, una herramienta de sincronización de archivos muy utilizada en entornos Unix/Linux. Este enfoque se realiza en tres pasos:

1. Ejecutar un primer `rsync` mientras el servidor está en funcionamiento, esta copia no será consistente, pero ya transferirá la mayor parte de los datos.
2. Apagar el servidor temporalmente.
3. Ejecutar un segundo `rsync` con la opción `--checksum` para sincronizar únicamente los archivos modificados.

El segundo paso será más rápido, ya que solo se copiarán los cambios desde el primer `rsync`, esto reduce significativamente el tiempo de inactividad requerido para la copia de seguridad. Es importante utilizar `--checksum`, ya que `rsync` solo detecta cambios en archivos con diferencia de un segundo en su marca de tiempo, y eso puede ser insuficiente para detectar cambios menores.

Consideraciones adicionales:

- Aunque este tipo de copia de seguridad puede ser más rápida que un volcado lógico (`pg_dump`), generalmente ocupa más espacio en disco. Esto se debe a que los archivos físicos incluyen todo el contenido de los índices y estructuras internas, mientras que `pg_dump` omite estos datos y los reemplaza por comandos para recrearlos.



- A pesar de sus desventajas, la copia de seguridad a nivel del sistema de archivos puede ser útil en entornos donde el tiempo de respaldo debe ser muy corto y donde los procedimientos de snapshot o sincronización están bien establecidos.

3. Archivar continuo y recuperación en un momento dado (PITR)

PostgreSQL incorpora un mecanismo robusto de protección contra fallos basado en el registro de escritura anticipada, conocido como WAL (Write-Ahead Logging). Este sistema registra en disco cada cambio realizado en los datos antes de aplicarlo realmente a los archivos de la base de datos. Gracias a este registro, si el servidor falla de forma inesperada (por ejemplo, por un corte de energía o un error del sistema operativo), es posible reproducir los cambios pendientes y recuperar automáticamente un estado coherente al reiniciar.

Este mismo principio que permite la recuperación ante fallos también se puede aprovechar para implementar un modelo de respaldo más poderoso y flexible: el archivado continuo (continuous archiving), junto con la recuperación en un momento dado (Point-In-Time Recovery o PITR).

¿En qué consiste el archivado continuo?

El archivado continuo combina una copia base del sistema de archivos (igual que la explicada en la sección anterior) con el respaldo sistemático de los archivos WAL generados a medida que la base de datos opera. Al mantener una copia inicial del clúster y todos los segmentos WAL posteriores, es posible reproducir todos los cambios ocurridos desde la copia base hasta el momento que se desee restaurar, incluso con precisión de segundos. Este enfoque permite restaurar el estado de la base de datos en cualquier punto del tiempo cubierto por la copia base y los WAL archivados. Es particularmente útil en situaciones donde, por ejemplo, se ejecutó por error una eliminación de datos y se desea volver a un estado justo anterior.

Beneficios del PITR

- No necesitamos una copia de seguridad del sistema de archivos perfectamente consistente como punto de partida. Cualquier inconsistencia interna en la copia física será corregida mediante la reproducción del WAL, de modo similar a cómo funciona la recuperación tras un fallo. Por lo tanto, no dependemos estrictamente de snapshots sofisticadas, una herramienta de archivo como tar puede servir, siempre que se complemente con WAL archivado correctamente.
- Dado que podemos combinar una secuencia continua de archivos WAL para su reproducción, se logra una copia de seguridad efectivamente continua y basta con seguir archivando WAL. Esto es especialmente valioso para bases de datos grandes, donde realizar copias físicas completas con frecuencia puede resultar costoso o disruptivo.
- No es obligatorio reproducir los WAL hasta el final de la secuencia. Se puede detener la reproducción en un momento deseado y obtener una instantánea consistente tal como estaba la base de datos en ese instante. A eso se refiere la "recuperación en un momento dado" (PITR). Podemos restaurar la base de datos al estado exacto en el tiempo especificado (dentro del rango cubierto por los WAL archivados).
- Si se alimentan continuamente los WAL archivados hacia una máquina secundaria (que tenga cargada la misma copia base), obtenemos un sistema de espera caliente (hot/warm standby) en cualquier momento, esa máquina puede ponerse en línea y tener una copia casi actual de la base de datos, lista para asumir el servicio si el servidor primario falla.



Limitaciones del método PITR

- **No permite restauraciones parciales:** Al igual que las copias físicas, el archivado continuo solo permite recuperar todo el clúster, no una sola base de datos, esquema o tabla.
- **Mayor consumo de almacenamiento:** Los WAL generados pueden ser numerosos, especialmente en sistemas con alta actividad transaccional, por lo que es necesario planificar bien la capacidad de almacenamiento.
- **Mayor complejidad operativa:** Requiere configurar correctamente la copia base, el archivado de WAL, monitoreo del proceso y pruebas regulares de restauración para garantizar que el procedimiento funcione correctamente en caso de desastre.

Configuración del archivado WAL

Un servidor PostgreSQL en funcionamiento produce indefinidamente una secuencia de registros WAL. Físicamente, esta secuencia se divide en segmentos WAL, que por defecto tienen un tamaño de 16 MB (aunque este valor puede cambiarse durante la inicialización con initdb). Cada segmento WAL tiene un nombre numérico que indica su posición en la secuencia. Cuando ciertos segmentos no son necesarios (por ejemplo, los anteriores al último punto de control), PostgreSQL puede "reciclarlos" reutilizándolos con nuevos nombres numéricos.

Cuando habilitamos el archivado de WAL, nuestra tarea es capturar el contenido completo de cada segmento una vez que esté finalizado, y copiarlo a un almacenamiento externo (otro disco, servidor, cinta, almacenamiento en la nube, etc.) antes de que el segmento sea reciclado. El método exacto de almacenamiento queda a criterio del administrador, podría usarse NFS, unidades de cinta, medios ópticos o cualquier otro sistema que permita recuperar los archivos más tarde.

Para otorgar flexibilidad, PostgreSQL no impone un método único de archivado, sino que permite que el administrador especifique un comando de shell o una biblioteca personalizada (módulo en C) que se ejecutará para copiar cada segmento terminado. Para habilitar el archivado WAL en PostgreSQL, deben ajustarse algunos parámetros en el archivo `postgresql.conf`.

Estos son:

- `wal_level = replica`
- `archive_mode = on`
- `archive_command = 'cp %p /ruta/destino/%f'`

Donde:

- `wal_level`: Se debe establecer al menos en `replica` para que el sistema genere suficientes detalles en los WAL.
- `archive_mode`: Activa el archivado.
- `archive_command`: Comando que será ejecutado automáticamente cada vez que un archivo WAL se complete. `%p` representa la ruta completa al archivo y `%f` su nombre.

Un ejemplo típico en sistemas Unix podría ser:

```
archive_command = 'test ! -f /backups/wal/%f && cp %p /backups/wal/%f'
```

Este comando copia el archivo solo si no existe previamente en la carpeta de destino, evitando sobrescrituras accidentales.



En sistemas Windows, un comando equivalente sería:

```
archive_command = 'copy "%p" "C:\\backups\\wal\\%f"'
```

los ejemplos anteriores aseguran que solo se copie el segmento si no existe aún en el directorio de destino, evitando sobrescrituras innecesarias. Cuando PostgreSQL necesite archivar un archivo, reemplazará %p y %f por valores reales.

Por ejemplo, puede generar:

```
test ! -f /mnt/server/archivedir/00000001000000A9000000065 && cp  
pg_wal/00000001000000A9000000065  
/mnt/server/archivedir/00000001000000A9000000065
```

Este comando se ejecutará para cada segmento WAL que esté listo para archivar. El comando de archivado se ejecuta bajo el mismo usuario que corre el servidor PostgreSQL, dado que los segmentos WAL contienen prácticamente toda la información del historial de la base de datos, es importante protegerlos contra accesos no autorizados (por ejemplo, directorios que no admitan lectura pública).

Debe garantizarse que el comando retorne un estado de salida cero solo cuando la copia sea exitosa. Si retorna otro valor, PostgreSQL asumirá que el archivado falló y volverá a intentar más tarde. También es posible usar un módulo de archivado personalizado (archive_library), el cual ofrece más rendimiento o funcionalidades específicas, aunque requiere escribir código en C y es más complejo de preparar.

Cuando el comando de archivo finaliza por una señal (que no sea SIGTERM para el apagado) o por un error del shell con un estado mayor que 125 (por ejemplo, comando no encontrado) o si la función de archivado emite un error, el proceso de archivado se cancela y el sistema lo vuelve a intentar periódicamente. En esos casos, los errores no se reportan en pg_stat_archiver.

Los módulos de archivo deben diseñarse para no sobrescribir archivos ya existentes, si existiera un archivo con el mismo nombre, el módulo debe comparar su contenido con el archivo que se pretende archivar. Si son idénticos, puede retornar éxito, si difieren debe retornar error. Esto evita corrupciones por sobrescrituras accidentales de WAL y es recomendable probar exhaustivamente la solución de archivado para este comportamiento.

En situaciones excepcionales, PostgreSQL puede intentar re-archivar un segmento WAL que ya había sido archivado (por ejemplo, si el sistema falla antes de que el servidor registre exitosamente el archivado). En ese caso, el módulo o comando debe verificar si el archivo ya existe y actuar apropiadamente (aceptar si idéntico, rechazar si distinto).

Un detalle delicado: el comando de ejemplo Unix evita sobrescrituras mediante un paso test ! -f En algunas plataformas Unix, cp -n o cp --no-clobber podrían parecer útiles, pero no se debe confiar plenamente sin asegurarse de que el estado de salida sea manejado correctamente. Por ejemplo, GNU cp -n retorna estado cero incluso si el archivo destino ya existe, lo cual no es deseable en este contexto.

Al diseñar la estrategia de archivado, hay que prever qué sucede si el comando de archivado falla repetidamente (por problemas de espacio, fallos en la cinta, etc.). El directorio pg_wal/ seguirá acumulando segmentos WAL hasta que se resuelva el problema, y si el disco se llena, PostgreSQL generará un cierre de pánico (PANIC). En ese escenario no se pierden transacciones confirmadas, pero la base de datos dejará de aceptar conexiones hasta que se libere espacio o corrija el error.



La velocidad del comando de archivado no tiene que ser instantánea, pero sí debe acompañar el ritmo promedio de generación de WAL. Si el archivador se queda rezagado de forma prolongada, habrá mayor riesgo de saturar pg_wal/ y de perder posibilidades de recuperación. Por ello, es fundamental monitorear el proceso de archivado constantemente.

Los nombres de los archivos WAL pueden tener hasta 64 caracteres y contener letras, dígitos y puntos ASCII. No es obligatorio conservar la ruta completa (%p), pero sí el nombre (%f) en la copia de seguridad.

Aunque el archivado WAL posibilita restaurar prácticamente todas las modificaciones en los datos, no restaura los cambios realizados en los archivos de configuración de PostgreSQL (como postgresql.conf, pg_hba.conf, pg_ident.conf), ya que estos no se modifican mediante comandos SQL. Es recomendable mantener esos archivos en respaldos separados vía copias físicas del sistema de archivos.

El comando o función de archivado solo se invoca cuando un segmento WAL está completo, en servidores con poca actividad, pueden pasar períodos prolongados sin generar segmentos completos. Para limitar la antigüedad de los WAL no archivados, se dispone del parámetro archive_timeout, que fuerza la creación de un nuevo segmento después de un tiempo máximo. Sin embargo, es importante no configurarlo demasiado bajo, pues esto puede inflar excesivamente el almacenamiento de WAL archivados.

También se puede forzar manualmente el cambio de segmento a través de:

```
SELECT pg_switch_wal();
```

Esto puede ser útil justo después de transacciones críticas que se desean archivar inmediatamente.

Algunos comandos SQL están optimizados para evitar escribir en WAL (dependiendo del wal_level). Si durante su ejecución el archivado o la replicación se encuentran activos, puede que no haya suficiente información en el WAL para recuperar esos cambios si no se consideró al configurar el wal_level. Por esta razón, wal_level solo puede configurarse al inicio del servidor (no en caliente). Sin embargo, el archive_command sí puede cambiarse mediante una recarga de configuración.

Recuperación usando PITR

Para recuperar una base de datos usando archivado continuo y restaurar a un punto específico en el tiempo, se deben seguir estos pasos:

1. Detener el servidor PostgreSQL (si está en ejecución).
2. Eliminar o renombrar el directorio de datos actual.
3. Restaurar la copia base del sistema de archivos.
4. Copiar los archivos WAL archivados a una ubicación accesible.
5. Crear un archivo recovery.signal en el directorio de datos.
6. Editar postgresql.conf o un archivo postgresql.auto.conf para incluir:
 - restore_command = 'cp /ruta/wal/%f %p'
 - recovery_target_time = '2025-10-04 14:30:00' # o un tiempo válido
7. Iniciar PostgreSQL, que automáticamente comenzará a aplicar los archivos WAL desde la copia base hasta el punto indicado.

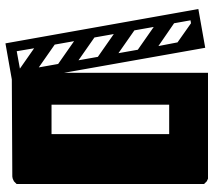
Una vez finalizada la recuperación, PostgreSQL sale del modo de recuperación y continúa operando normalmente.



Consideraciones finales sobre PITR

- `pg_dump` y `pg_dumpall` no producen copias físicas ni contienen la información necesaria para que WAL pueda reproducirse. Son volcados lógicos, adecuados para migraciones o copias de datos, pero no sirven como parte de una solución de archivado continuo.
- Al igual que la copia física simple, el método con WAL solo puede restaurar el conjunto completo del clúster, no bases de datos individuales ni tablas aisladas.
- Este enfoque demanda mucho espacio de almacenamiento, la copia base puede ser voluminosa, y en sistemas activos se generará gran cantidad de WAL que deben respaldarse. Aun así, para entornos donde se requiere alta confiabilidad y mínima pérdida de datos, es una técnica de respaldo muy preferida.

Para que la recuperación funcione correctamente, es esencial mantener una secuencia continua de WAL archivados desde el momento en que se realizó la copia base inicial. Por ello, antes de hacer cualquier copia base, se debe configurar y probar el mecanismo de archivado WAL. Lo primero es comprender cómo PostgreSQL maneja los WAL internamente.



Actividad Interactiva #1

Mira el video del link presentado a continuación y escribe en las líneas de abajo un resumen de 4 líneas de lo que consideres más importante.

<https://www.youtube.com/watch?v=sP6Ejpgahg8&t=158s>

Sección 2: Comandos para recuperación.

La recuperación de bases de datos en PostgreSQL se puede llevar a cabo de dos maneras principales:

- Restaurando un volcado lógico (creado con `pg_dump` o `pg_dumpall`) con `psql` o `pg_restore`.
- Usando una copia física completa más archivos WAL para una recuperación punto en el tiempo (PITR).

A continuación, se explican ambos métodos detalladamente:

Restaurando el volcado (Restoring the Dump)

Los archivos de texto creados por `pg_dump` están diseñados para ser utilizados por la herramienta `psql`, ya que contienen un conjunto de instrucciones SQL que deben ejecutarse dentro de una base de datos PostgreSQL para restaurar su contenido. La forma más sencilla y común de restaurar un volcado lógico es utilizando un comando del tipo:

```
psql nombrebd < archivo_volcado
```

Donde: **archivo_volcado** es el archivo generado previamente por `pg_dump`.

Es importante tener en cuenta que este comando no crea la base de datos de destino (`nombrebd`), por lo que el usuario debe asegurarse de crearla previamente. Lo recomendable es hacerlo a partir de la plantilla `template0`, para evitar que se incluyan objetos adicionales no deseados provenientes de `template1`. Esto se logra, por ejemplo, con un comando como `createdb -T template0 nombrebd`.



Al igual que `pg_dump`, `psql` acepta diversas opciones para especificar el servidor al cual conectarse, el nombre de usuario que se desea utilizar, entre otras configuraciones. La documentación de `psql` proporciona una referencia completa de estas opciones para facilitar el proceso.

En los casos donde el archivo generado por `pg_dump` no es de texto (es decir, cuando se utiliza el formato personalizado `-Fc` o el formato `tar -Ft`), debe usarse la herramienta `pg_restore` para llevar a cabo la restauración. Esta herramienta ofrece mayor flexibilidad, ya que permite restaurar objetos específicos, omitir errores y aplicar filtros durante la restauración.

Antes de iniciar la restauración de un volcado en formato SQL, es necesario asegurarse de que existan todos los usuarios y roles que originalmente poseían objetos o contaban con permisos dentro de la base de datos respaldada. De lo contrario, `psql` no podrá recrear estos objetos con las propiedades y privilegios originales, lo cual podría afectar la integridad o funcionalidad del sistema restaurado. Aunque en ciertos escenarios puede ser útil modificar la propiedad de los objetos, por lo general se busca replicar fielmente la configuración previa.

De manera predeterminada, cuando se encuentra con un error de SQL durante la ejecución del script de restauración, `psql` continúa procesando las siguientes instrucciones, lo que puede llevar a restauraciones incompletas o inconsistentes sin que el usuario se percate de ello. Para evitar esto, se recomienda utilizar la variable `ON_ERROR_STOP`, la cual indica a `psql` que detenga la ejecución del script y devuelva un estado de error si se encuentra algún fallo.

Esto puede hacerse mediante el siguiente comando:

```
psql --set ON_ERROR_STOP=on nombrebd < archivo_volcado.
```

Esta configuración permite detectar errores de forma inmediata y evitar restauraciones parciales que podrían comprometer el sistema.

Otra alternativa disponible es realizar la restauración dentro de una única transacción, usando la opción `-1` o `--single-transaction`. Esto asegura que todo el contenido del volcado se aplique completamente o, en caso de error, se revierta en su totalidad, garantizando así la consistencia de la base de datos restaurada. No obstante, este enfoque implica que cualquier error, incluso menor, anulará todo el proceso, lo cual puede ser problemático si la restauración es extensa y ha tomado varias horas. Aun así, puede resultar preferible frente al esfuerzo de limpiar manualmente una base de datos que ha sido restaurada de forma incompleta.

Tanto `pg_dump` como `psql` permiten el uso de tuberías, lo que hace posible realizar una transferencia directa entre servidores sin necesidad de almacenar el archivo intermedio. Por ejemplo, se puede ejecutar `pg_dump -h host1 nombrebd | psql -h host2 nombrebd` para clonar una base de datos directamente entre dos servidores, lo que resulta especialmente útil en entornos de migración o pruebas.

Es importante destacar que los volcados generados por `pg_dump` asumen una base limpia basada en `template0`. Esto significa que si durante la creación de la base de datos se utilizó una plantilla personalizada (`template1`), es posible que se generen conflictos al intentar restaurar funciones, extensiones o configuraciones duplicadas. Para evitar estos problemas, se recomienda que la base de datos vacía que se utilizará como destino de la restauración sea creada desde `template0`, tal como se indicó anteriormente.



Una vez finalizada la restauración, es recomendable ejecutar el comando ANALYZE sobre la base de datos. Esta operación permite que el optimizador de consultas recoja estadísticas actualizadas sobre la distribución de los datos, lo cual es esencial para generar planes de ejecución eficientes en futuras consultas SQL.

Recuperación mediante una copia de seguridad de archivo continua

En los casos donde se realiza una copia de seguridad física completa, que incluye los archivos de la base de datos y el registro de transacciones (WAL), la recuperación se basa en restaurar estos archivos en el sistema de archivos y permitir que PostgreSQL procese los archivos WAL para alcanzar un estado consistente o incluso un momento específico en el pasado.

Ante una situación de pérdida o daño, el primer paso consiste en detener el servidor de base de datos si está en ejecución. A continuación, si el espacio lo permite, se recomienda hacer una copia del directorio completo del clúster de datos y de los tablespaces, como medida de precaución en caso de que se necesite recuperar información adicional. Esta precaución requiere suficiente espacio en disco para duplicar el tamaño de la base de datos. En situaciones donde esto no sea posible, se debe al menos preservar el contenido del subdirectorio pg_wal, ya que podría contener archivos WAL no archivados previamente al fallo del sistema.

Luego, se deben eliminar todos los archivos y subdirectorios existentes en el directorio de datos del clúster y en los directorios raíz de los tablespaces utilizados. Posteriormente, se procede a restaurar los archivos de la base de datos desde la copia de seguridad del sistema de archivos, asegurándose de mantener los permisos y el propietario correctos. Es fundamental que los archivos restaurados pertenezcan al usuario del sistema bajo el cual opera PostgreSQL, y no al usuario root. En el caso de utilizar tablespaces, también se debe verificar que los enlaces simbólicos ubicados en el directorio pg_tblspc/ hayan sido restaurados correctamente.

Después de esto, se debe limpiar el contenido de pg_wal/, ya que estos archivos provienen de una copia previa y podrían estar desactualizados. Si no se respaldó este directorio, debe recrearse con los permisos adecuados, asegurándose de mantenerlo como un enlace simbólico si así estaba configurado previamente.

Si se guardaron archivos WAL desarchivados durante el proceso de respaldo, estos deben copiarse en el directorio pg_wal/, evitando moverlos directamente para conservar los originales en caso de que sea necesario reiniciar el proceso de recuperación desde cero.

El siguiente paso es establecer los ajustes necesarios en el archivo postgresql.conf, particularmente el parámetro restore_command, y crear un archivo vacío llamado recovery.signal en el directorio de datos del clúster. También se recomienda modificar temporalmente el archivo pg_hba.conf para evitar conexiones de usuarios normales hasta verificar que la recuperación fue exitosa.

Una vez configurado todo, se puede iniciar el servidor. PostgreSQL entrará en modo de recuperación y comenzará a procesar los archivos WAL archivados. Si la recuperación se interrumpe por algún error externo, basta con reiniciar el servidor, el cual continuará automáticamente el proceso de recuperación desde el último punto consistente. Cuando la recuperación finalice correctamente, el archivo recovery.signal será eliminado automáticamente y el servidor pasará a operar en modo normal.



Tras este proceso, se debe inspeccionar cuidadosamente el contenido de la base de datos para verificar que se ha alcanzado el estado deseado. Si no es así, se puede repetir el procedimiento desde el principio. Si todo es correcto, se puede permitir nuevamente el acceso de los usuarios restaurando el archivo `pg_hba.conf` a su configuración original.

El parámetro `restore_command` indica a PostgreSQL cómo recuperar archivos WAL desde su ubicación de archivo. Puede definirse como una instrucción de shell que incluya las variables `%f` (nombre del archivo WAL requerido) y `%p` (ruta de destino dentro del sistema). Por ejemplo, una instrucción simple puede ser:

```
restore_command = 'cp /mnt/server/archivedir/%f %p'.
```

Esta instrucción copia los archivos desde un directorio donde fueron previamente archivados. Es posible, por supuesto, definir comandos más complejos, incluyendo scripts que soliciten la intervención del operador o que interactúen con soluciones de almacenamiento más avanzadas.

Es fundamental que este comando devuelva un estado de salida distinto de cero cuando no se encuentra un archivo requerido, ya que esto no se considera un error crítico. Sin embargo, si el comando falla debido a una señal inesperada o un error del shell, la recuperación será cancelada y el servidor no podrá iniciar. El sistema también puede solicitar archivos adicionales, como los que tienen el sufijo `.history`, lo cual es normal en ciertas situaciones de recuperación.

En condiciones normales, la recuperación procesa todos los segmentos WAL disponibles hasta alcanzar el punto más reciente posible. Esto finaliza con un mensaje que indica que no se pudo encontrar un archivo, lo cual es esperable. Si se desea recuperar la base de datos hasta un momento anterior específico, por ejemplo, antes de un error humano, se puede establecer un "objetivo de recuperación" (recovery target), especificando una fecha/hora, un punto de restauración con nombre, o (menos comúnmente) un ID de transacción. De estas opciones, las más utilizadas son las dos primeras, ya que no existen herramientas prácticas para identificar transacciones individuales.

Cabe señalar que el objetivo de recuperación debe ubicarse después del punto en que se completó la copia de seguridad base (el momento de finalización de `pg_backup_stop`). No es posible utilizar una copia de seguridad base para restaurar el estado de la base de datos durante el tiempo en que se realizó dicha copia. En ese caso, se debe recurrir a una copia anterior.

En caso de que se detecten datos WAL corruptos, la recuperación se detendrá y el servidor no arrancará. Si el problema fue externo, como una falla del sistema, puede reiniciarse el proceso de recuperación. Si la corrupción está dentro del archivo, se puede volver a comenzar desde una copia base más antigua y aplicar un objetivo de recuperación anterior al punto de corrupción.

Sección 3: Migración de la Base de Datos

La migración de una base de datos PostgreSQL es un proceso común cuando se necesita trasladar información entre entornos, por ejemplo, desde un servidor de desarrollo a producción, entre proveedores de servicios en la nube, o simplemente como parte de un respaldo estructurado. Este procedimiento puede realizarse mediante exportación e importación utilizando herramientas proporcionadas por PostgreSQL como `pg_dump`, `psql` o `pg_restore`, dependiendo del tipo de archivo y de los requerimientos del entorno.



Importar una Base de Datos PostgreSQL

Para importar una base de datos previamente exportada en formato SQL (es decir, como un archivo de texto con instrucciones SQL que recrean la estructura y los datos), se puede utilizar el comando `psql`. Este comando ejecuta el contenido del archivo `.sql` directamente sobre una base de datos vacía o existente.

La sintaxis básica del comando es:

```
psql -h EXTERNAL_HOSTNAME -U USERNAME -p PORT -d DATABASE_NAME < export.sql
```

En este comando, deberás reemplazar:

- `EXTERNAL_HOSTNAME`: por la dirección del servidor al que deseas conectarte.
- `USERNAME`: por el nombre del usuario de la base de datos.
- `PORT`: por el número de puerto utilizado por PostgreSQL (por defecto suele ser 5432).
- `DATABASE_NAME`: por el nombre de la base de datos destino.
- `export.sql`: por la ruta y nombre del archivo SQL que deseas importar.

Al ejecutar este comando, el sistema solicitará la contraseña del usuario especificado, y si todo está correcto, el contenido del archivo comenzará a ejecutarse en la base de datos indicada. Cabe destacar que no se mostrará ninguna salida si no hay errores, por lo que es posible que el proceso parezca silencioso. Sin embargo, cualquier error será mostrado directamente en la línea de comandos.

Una vez finalizada la importación, es recomendable verificar que los objetos (tablas, vistas, funciones, etc.) se han creado correctamente. Esto se puede hacer conectándose a la base de datos mediante `psql` y ejecutando comandos como `\dt` (para ver tablas) o consultando directamente algunos datos con instrucciones `SELECT`.

Nota importante: Si el archivo que estás intentando importar no es un archivo de texto plano SQL, sino un archivo en formato especial como `.dump` o `.tar`, entonces no debes utilizar `psql`, ya que no podrá interpretar este tipo de archivo. En esos casos, deberás utilizar la herramienta `pg_restore`, que está diseñada para manejar archivos de respaldo en formato binario o `tar`.

Exportar una Base de Datos PostgreSQL

El primer paso para migrar una base de datos es exportarla desde el entorno original. PostgreSQL proporciona la herramienta `pg_dump` para este propósito, la cual permite volcar todo el contenido de una base de datos (estructura y datos) a un archivo de salida. Este archivo puede ser utilizado posteriormente para restaurar la base de datos en otro entorno o servidor.

El comando básico para exportar una base de datos en formato SQL es:

```
pg_dump -h EXTERNAL_HOSTNAME -U USERNAME -p PORT -d DATABASE_NAME > export.sql
```

En este comando también se deben sustituir los valores según corresponda:

- `EXTERNAL_HOSTNAME`: dirección del servidor de la base de datos a exportar.
- `USERNAME`: nombre del usuario con permisos de lectura.
- `PORT`: puerto del servidor PostgreSQL.



- DATABASE_NAME: nombre de la base de datos que deseas exportar.
- export.sql: nombre del archivo en el cual se guardará el volcado de la base de datos.

Este comando generará un archivo de texto plano .sql que contiene todas las instrucciones necesarias para recrear la base de datos en otro lugar. Al ejecutarse, pg_dump también solicitará la contraseña del usuario correspondiente.

Una vez finalizada la exportación, el archivo estará disponible en la ruta y con el nombre especificado en el comando. Puedes revisar su contenido abriéndolo con cualquier editor de texto plano (como Notepad++, Sublime Text, Visual Studio Code, entre otros), o también puedes utilizar una herramienta gráfica para bases de datos, como TablePlus, DBeaver o PgAdmin, que permiten analizar fácilmente su contenido.

Nota: Si estás trabajando con una base de datos de gran tamaño, puede ser más eficiente exportarla en un formato de almacenamiento comprimido o binario, como .dump o .tar. Estos formatos reducen el tamaño del archivo resultante y hacen más rápida la exportación e importación. Para revisar el contenido de este tipo de archivos, no basta con abrirlos en un editor de texto, sino que deben ser restaurados usando pg_restore.

Actividad Interactiva #2

Ejercicio de asociación.

A continuación, verás una lista de términos y una lista de definiciones. Asocia Cada Término con su Definición:
Une cada término con la definición correcta.

Términos:

1. pg_dump
2. pg_restore
3. Copia de seguridad continua (backup continuo)
4. Archivo WAL
5. archive_command
6. Punto de recuperación (Recovery Target)

Definiciones:

- A. Proceso que mantiene una copia constante y actualizada de los archivos de la base de datos para recuperación ante fallos.
- B. Utilidad para exportar la base de datos a un archivo de texto o volcado.
- C. Herramienta para restaurar volcados en formatos personalizados o comprimidos generados por pg_dump.
- D. Registro de escritura anticipada que contiene los cambios realizados en la base de datos para recuperación y replicación.
- E. Configuración que permite definir la ubicación y método para archivar archivos WAL.
- F. Técnica que permite restaurar la base de datos a un punto específico en el tiempo, usando un objetivo de recuperación.

Instrucciones: Asocia cada término (1-6) con la definición correspondiente (A - F). Y escríbela a continuación.

Ej: "1 – B"

